Name: Abhishek Pandey SAP ID: 60019220110
Roll No: B073 Branch: CSE(ICB)

EXPERIMENT 3

Aim: Deploy the public key infrastructure (PKI) with an Ethereum blockchain.

Theory:

1. Key Pair Generation in Ethereum & Private Key Security
Ethereum key pairs are generated using elliptic curve cryptography (specifically secp256k1). A private key is a randomly generated 256-bit number, and the corresponding public key is derived from it through elliptic curve multiplication. The Ethereum address is then derived from the last 20 bytes of the Keccak-256 hash of the public key. Security of the private key is critical because anyone who has it can access the funds or identity linked to that address. Best practices include storing private keys in hardware wallets, encrypting them with strong passwords, using secure key vaults, and never hard-coding or exposing them in source code.

---

2. Deploying a Smart Contract as a Certificate Authority (CA)
A CA smart contract on Ethereum is deployed like any other contract, written in Solidity and published through a transaction. Its core functions would include:

- `issueCertificate(address user, string details)` → to create and assign a certificate to a user.
- `revokeCertificate(uint certId)` → to revoke compromised or expired certificates.
- `verifyCertificate(uint certId)` → to check if a certificate is valid.
- `getCertificate(uint certId)` → to retrieve details of a stored certificate.
  Deployment ensures immutability, transparency, and accessibility for all users to trust the CA without relying on a centralized authority.

---

3. Issuing a Digital Certificate via Ethereum
When issuing a certificate, the CA smart contract records key details such as the user's public key, user identity information (name, org, or hashed identity), certificate ID, issue date, and expiration date. This data is stored on the Ethereum blockchain as a mapping or struct. The blockchain acts as a tamper-proof ledger, ensuring that once a certificate is issued, it cannot be altered or forged. Only authorized CA accounts can invoke the `issueCertificate` function to add a new certificate.

---

4. Verifying a Digital Certificate on Ethereum
Verification involves querying the smart contract to ensure the certificate exists, is active (not revoked/expired), and was issued by the trusted CA. The public key plays a critical role —

it's used by relying parties to validate signatures created with the certificate holder's private key, proving authenticity. The smart contract's verification function combined with cryptographic checks ensures that certificates are both genuine and tied to the correct public key.

---

5. Advantages & Challenges of Ethereum-based PKI
Advantages:

- **Decentralization:** Removes reliance on a single root CA; trust is distributed across the blockchain.
- **Transparency & Immutability:** Certificate issuance and revocation are publicly visible and tamper-proof.
- **Automation:** Smart contracts enforce policies without manual intervention.

Challenges:

- **Scalability:** High transaction volume for certificate issuance/revocation could cause network congestion.
- **Cost:** Each operation costs gas, making large-scale PKI more expensive than traditional systems.
- **Privacy:** Storing identity data directly on-chain risks exposure unless properly hashed/encrypted.
- **Adoption:** Existing systems and browsers are built around traditional CA hierarchies, so integration is non-trivial.

Code:

SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)

Name: Abhishek Pandey
Roll No: B073

SAP ID: 60019220110
Branch: CSE(ICB)

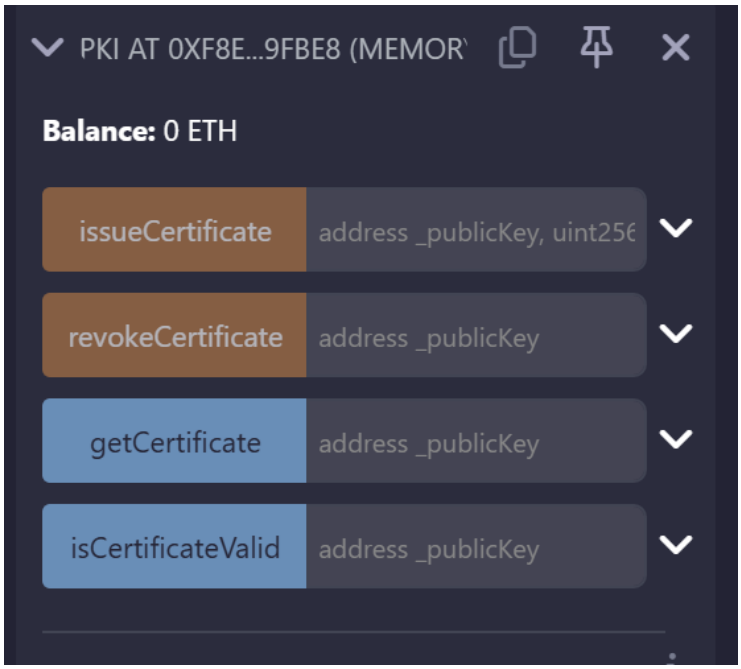
```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract PKI {
    // Certificate structure
    struct Certificate {
        address publicKey;   // Public key of the end entity
        address issuer;      // Address of the CA (issuer)
        uint256 expiryDate;  // Expiration date (Unix timestamp)
        bytes signature;     // Digital signature (for demo, just a placeholder)
        bool isRevoked;      // Status of the certificate (revoked or not)
    }

    // Mapping of user public key => certificate
    mapping(address => Certificate) private certificates;

    // Events
    event CertificateIssued(address indexed publicKey, address indexed issuer, uint
    event CertificateRevoked(address indexed publicKey);

    // Function to issue a certificate
    function issueCertificate(    infinite gas
        address _publicKey,
```

```
    function issueCertificate(    infinite gas
        address _publicKey,
        uint256 _expiryDate,
        bytes memory _signature
    ) public {
        require(certificates[_publicKey].publicKey == address(0), "Certificate alre
        certificates[_publicKey] = Certificate({
            publicKey: _publicKey,
            issuer: msg.sender,
            expiryDate: _expiryDate,
            signature: _signature,
            isRevoked: false
        });
        emit CertificateIssued(_publicKey, msg.sender, _expiryDate);
    }

    // Function to revoke a certificate
    function revokeCertificate(address _publicKey) public {    30514 gas
        require(certificates[_publicKey].publicKey != address(0), "Certificate not
        require(certificates[_publicKey].issuer == msg.sender, "Only issuer can rev
        certificates[_publicKey].isRevoked = true;
        emit CertificateRevoked(_publicKey);
    }
```

Name: Abhishek Pandey
Roll No: B073
SAP ID: 60019220110
Branch: CSE(ICB)

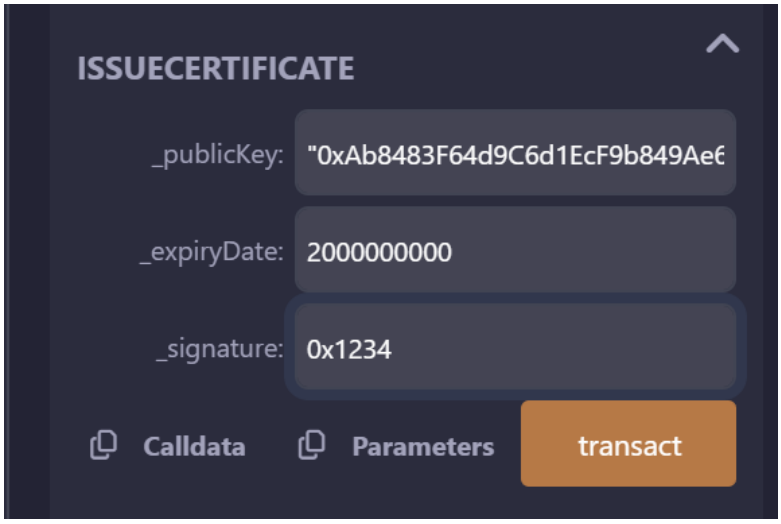
```
        emit CertificateRevoked(_publicKey);
    }

    // Function to get certificate details
    function getCertificate(address _publicKey) public view returns (    infinite ga
        address, address, uint256, bytes memory, bool
    ) {
        Certificate memory cert = certificates[_publicKey];
        require(cert.publicKey != address(0), "Certificate not found");
        return (cert.publicKey, cert.issuer, cert.expiryDate, cert.signature, cert.
    }

    // Function to check if a certificate is valid (not revoked and not expired)
    function isCertificateValid(address _publicKey) public view returns (bool) {
        Certificate memory cert = certificates[_publicKey];
        if (cert.publicKey == address(0)) return false; // not issued
        if (cert.isRevoked) return false;               // revoked
        if (block.timestamp > cert.expiryDate) return false; // expired
        return true;
    }
}
```

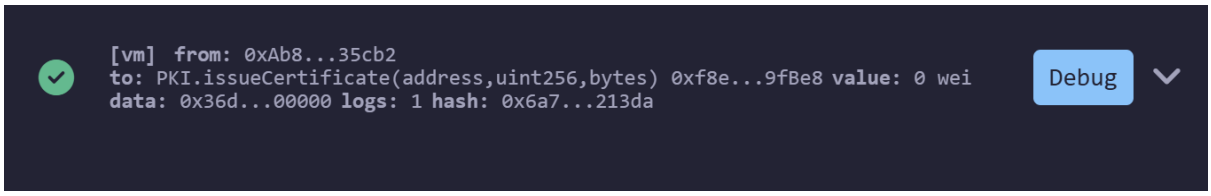
Output:



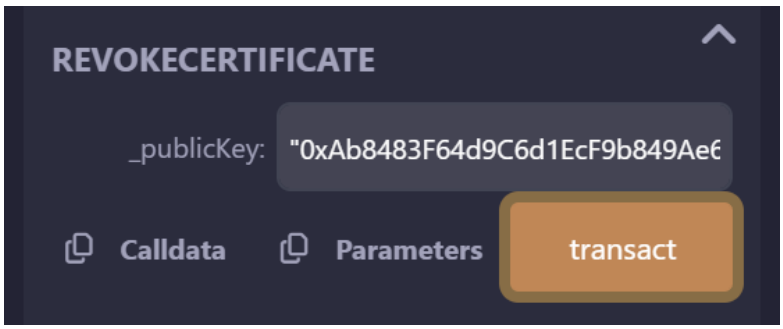
Issue:

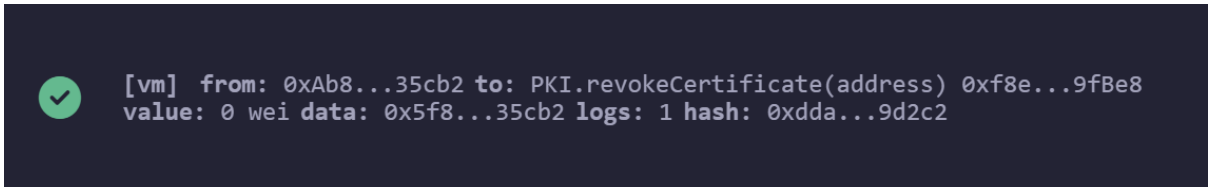
Name: Abhishek Pandey
Roll No: B073
SAP ID: 60019220110
Branch: CSE(ICB)





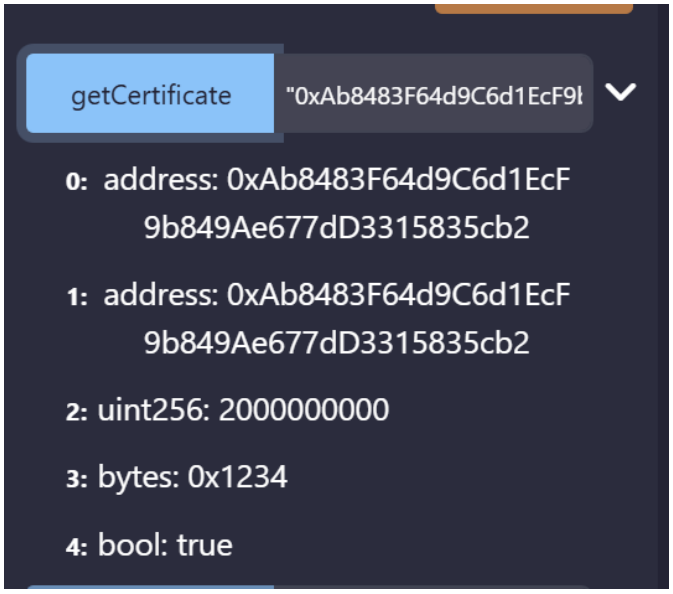
Revoke:



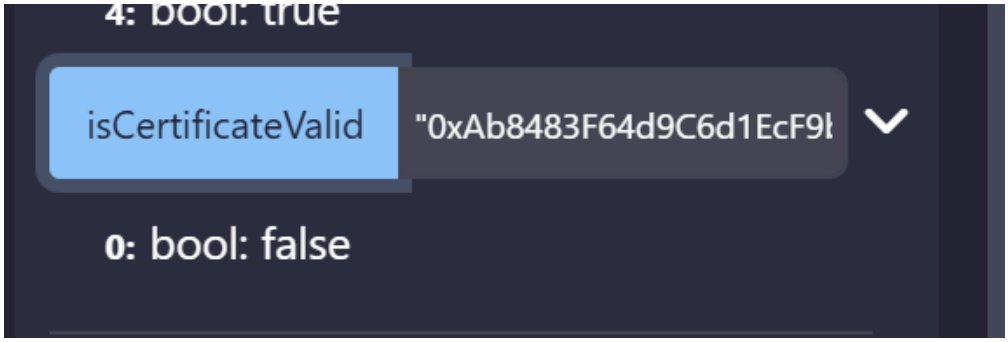


Get:

Name: Abhishek Pandey SAP ID: 60019220110
Roll No: B073 Branch: CSE(ICB)



isValid:



Conclusion:

Thus we successfully deployed the public key infrastructure (PKI) with an Ethereum blockchain.